

MINDPULSE AI

Student Mental Health Support & Analytics Platform

System Architecture & Detailed Module Explanations

About This Document

This document provides a complete technical analysis and explanation of all code modules comprising the MindPulse AI platform. It is structured to detail the architecture, database designs, backend API endpoints, analytical algorithms, machine learning risk engine, anonymous peer wall support system, and interactive Streamlit frontend layout. Emojis and source code references are detailed contextually.

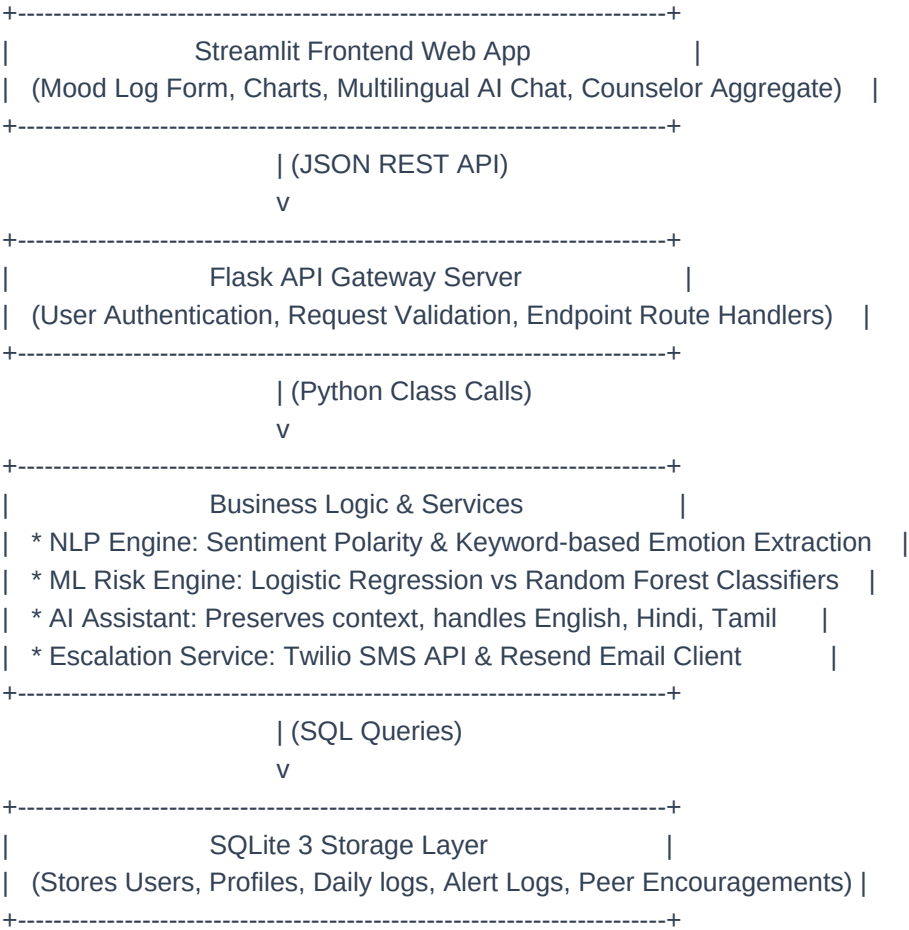
Document Metadata

System Version:	v1.1 (Production/Demo ready)
Generated Date:	June 26, 2026
Architecture Type:	Separated Monorepo (Flask Backend API + Streamlit Frontend UI)
Database Engine:	SQLite 3
Tech Stack Layers:	Python 3, Streamlit, Flask, SQLite, Scikit-learn, TextBlob, Plotly

1. Architecture Overview & Design Intent

MindPulse AI is designed to serve as an ethical, privacy-conscious student wellbeing support tool. It correlates behavioral patterns (attendance, assignment loads, social connectivity, sleep, exam pressure) with mood trends to detect early warning signs of stress or mental health struggles.

System Components Diagram



Separating the user interface (Streamlit) from the database and business logic (Flask API) ensures the app is robust, clean, and extensible. It allows counselors to access aggregate data via APIs without direct read permissions to individual database tables, protecting privacy.

2. Database Layer & Data Bootstrapping

The project uses SQLite 3 for database storage due to its lightweight footprint, making it ideal for academic projects, developer environments, and lightweight web service deployments. The database is structured to securely store passwords, daily mood logs, alert logs, and peer wall entries.

Database Schema Tables

- * **users:** Stores account credentials. Passwords are encrypted using cryptographically secure hashing (bcrypt or pbkdf2 with SHA256).
- * **profiles:** Stores student profile settings, including name, department, year, preferred language, anonymous status, and trusted contact details for alerts.
- * **checkins:** Logs daily student check-ins containing numerical metrics (mood_score, stress, energy, sleep, attendance, assignments, social, exam_pressure), NLP sentiments, and encrypted journal text.
- * **alerts:** Logs alert events (counselor triggers, trusted contact email/SMS dispatches) to maintain an audit trail of mental health escalations.
- * **peer_posts:** Logs anonymous posts, encouragement messages, and peer recovery stories posted to the community wall.
- * **buddy_matches:** Records pairs of students recommended for study pairings based on shared academic context and mutual buddy matching preferences.

Database Files & Utilities

File: backend/database.py
Purpose: Initializes and manages the SQLite database connections and table schemas.
Details: Defines standard context managers to connect, query, and close database handles safely. Contains DDL scripts to create database tables (users, profiles, checkins, alerts, peer_posts, etc.).

File: backend/data/seed.py
Purpose: Seeds simulated database records to demonstrate system features.
Details: Generates initial dummy accounts (including student and counselor profiles) and sets up mock student check-in history to render analytical patterns on dashboards immediately.

File: scripts/bootstrap_data.py
Purpose: A command-line script to initialize the project database.
Details: Can be run via 'python scripts/bootstrap_data.py' to wipe out old DB records, create fresh tables, and execute seed.py for initial demo usage.

3. Backend API Platform & Routing Gateway

The backend API acts as the core handler of student data, running on Flask. All frontend operations perform HTTP requests to access or modify resources, enforcing security, validation, and role-based permissions.

Core Backend Modules

File: backend/app_platform.py
Purpose: Configures the Flask API instance and error handling middleware.
Details: Handles JSON serialization, CORS configs (allowing frontend Streamlit communication), environment variables configuration, and request logging. Also runs the local development port binding.

File: backend/api/routes.py
Purpose: Defines all REST API endpoints used by the Streamlit application.
Details: Maps URL patterns to service methods. Key routes handle login/register, check-ins, user dashboard graphs, counselor summaries, peer support, Twilio/Resend alerts, and AI companion inputs.

File: shared/gateway.py
Purpose: Maintains API routing configurations and base gateway routes.
Details: Manages request routing to ensure the local frontend knows whether it is interacting with a local Flask service, a mock service, or a remote production backend on Render.

Key REST API Endpoints

- * POST /api/auth/register - Registers new users (hashes password).
- * POST /api/auth/login - Validates login, sets session context and user roles.
- * POST /api/checkins - Submits daily emoji check-in data, processes NLP scores, and saves logs.
- * GET /api/dashboard/<user_id> - Aggregates daily logs for visualization on the student's dashboard.
- * GET /api/admin/overview - Returns aggregated, anonymized student statistics strictly for counselors.
- * POST /api/assistant - Forwards journal prompts to the AI chatbot to fetch supportive guidance.

4. Business Logic & Support Services

The services folder contains the computational core of MindPulse AI. This layer performs Natural Language Processing on text, runs Machine Learning algorithms for risk analysis, orchestrates the conversational companion, and interfaces with communication APIs.

NLP Sentiment & Emotion Engine

File: backend/services/nlp_engine.py
Purpose: Extracts emotional metrics and sentiment ratings from the user's journal notes.
Details: Uses TextBlob to measure polarity (-1.0 to 1.0) and subjectivity (0.0 to 1.0). Implements a VADER-style rule matrix to compute compound sentiment scores. Extracts keyword-based emotions (e.g. Joy, Sadness, Anger, Fear, Anxiety) by cross-referencing text against emotional word banks.

Supervised ML Risk Engine

MindPulse AI classifies students' wellness risk into Normal, Stressed, and High Risk. Unlike basic apps using fixed rules, this system uses supervised Machine Learning.

- * **Dataset:** Trained on 1,800 simulated student wellness profiles to ensure data transparency.
- * **Features:** Uses 9 dimensions (mood score, stress levels, sleep hours, attendance, assignments, NLP compound sentiment, social connectedness, energy, and exam pressure).
- * **Supervised Classifier Comparison:** Compares a Logistic Regression model (serving as a linear baseline) and a Random Forest Classifier (serving as a non-linear, high-accuracy ensemble model).
- * **Model Selection:** Automatically calculates accuracy, precision, recall, and macro-F1 score on test splits, deploying the model with the highest F1 score.

File: backend/services/risk_engine.py
Purpose: Orchestrates dataset creation, model preprocessing, training, comparison, and inference.
Details: Splits data into an 80/20 train-test ratio, runs MinMaxScaler preprocessing, compares LogisticRegression and RandomForestClassifier, and saves/restores the best model for live check-in scoring.

Multilingual AI Support Companion

The AI companion provides conversational support using LLM prompting. It acts as a wellness partner rather than a diagnostic tool, adhering to ethical boundaries.

- * Multilingual Capability: Supports prompt and response styling in English, Hindi, and Tamil.
- * Ethical Guardrails: Explicitly instructed to avoid clinical diagnoses, suggest real-world campus counselors for heavy topics, and provide positive coping habits.
- * Context Preservation: Stores chat history context in the SQLite checkins journal thread to preserve conversational flow across interactions.

File: backend/services/assistant.py

Purpose: Configures AI system prompts, structures multilingual prompts, and handles chatbot responses.

Details: Interfaces with external generative API nodes using configured API keys and formats user inputs into appropriate, safety-filtered responses.

Escalation & Live Notification Alerts

When the risk engine classifies a student as 'High Risk', the system triggers an escalation warning. If the student has saved contact details and enabled alert permissions, notifications are sent immediately.

- * Email Alerts via Resend: Connects to the Resend API to deliver mental health alert emails to a trusted mentor or campus counselor.
- * SMS Alerts via Twilio: Interfaces with the Twilio SMS Gateway to dispatch real-time emergency alert texts.

File: backend/services/alerts.py

Purpose: Validates consent preferences and delivers email/SMS alerts using external communication APIs.

Details: Checks profile flags (ALERT_REAL_DELIVERY_ENABLED) and executes HTTP API requests to Twilio and Resend when high-risk scenarios are detected.

5. Frontend Web Application (Streamlit)

The frontend is built using Streamlit, providing an interactive dashboard. It features dynamic charts, clean metrics boxes, and a mobile-friendly responsive layout.

Authentication & Layout Core

File: frontend/Home.py
Purpose: The main entry point of the Streamlit application.
Details: Renders the login and registration forms. Implements role-aware session routing: directs students to the check-in and dashboard pages, and counselors to the insights portal.

File: frontend/ui.py
Purpose: Defines the visual design system, custom CSS, and shared layouts.
Details: Implements premium aesthetic adjustments (glassmorphism details, styling classes), custom CSS styles, emojis grids for the mood selection form, and helper metric boxes.

Interactive Pages

File: frontend/pages/1_Daily_Check_In.py
Purpose: Renders the daily check-in form where students submit emotional indicators.
Details: Includes mood selections, sleep sliders, attendance percentages, and text fields for journals. Submits inputs to Flask, runs the NLP/ML engines, and displays coping suggestions immediately.

File: frontend/pages/2_Mood_Dashboard.py
Purpose: Renders wellness history graphs using Plotly.
Details: Visualizes mood/stress correlations over time. Features a trend forecaster which uses simple LinearRegression on the user's past 15 days to forecast potential upcoming low-mood days.

File: frontend/pages/3_AI_Support_Companion.py
Purpose: Renders the interactive chat layout for the AI support partner.
Details: Displays message history and filters input languages based on user settings.

File: frontend/pages/4_Resource_Hub.py
Purpose: Lists campus resources, helplines, journal prompts, and coping tools.
Details: Provides phone numbers, location information, and interactive coping tools.

File: frontend/pages/5_Peer_Support.py
Purpose: Renders the community wall and buddy matcher.
Details: Displays anonymous recovery stories and academic buddy matches (pairing students with similar academic loads for mutual support).

File: frontend/pages/6_Counselor_Insights.py
Purpose: Renders anonymized aggregate metrics for administrative staff.
Details: Displays campus stress distribution metrics and risk counts without exposing private journals.

6. Test Suite & Verification

MindPulse AI includes a test suite running on pytest. This validates API endpoints, risk engine accuracy, and alerting logic to prevent deployment errors.

File: tests/conftest.py
Purpose: Configures shared pytest fixtures and temporary test databases.
Details: Sets up mock Flask clients and temporary SQLite instances to ensure tests run in isolation without modifying live data.

File: tests/test_api.py
Purpose: Validates core API routes, response formats, and status codes.
Details: Tests endpoints such as registration, logging check-ins, dashboard feeds, and buddy matching.

File: tests/test_platform.py
Purpose: Verifies system initialization and server-wide configurations.
Details: Checks environment variable parsing, database migration checks, and server CORS configs.

File: tests/test_auth_alerts.py
Purpose: Validates hashing security, role permissions, and alert triggers.
Details: Ensures password validation works, non-counselors are blocked from the admin portal, and alert dispatches are triggered correctly for high-risk flags.